

Deliverable 2's Report

Alberto Cimmino
Università Di Trento

Abstract—This paper evaluates two multi-GPU implementations of Sparse Matrix–Vector Multiplication (SpMV) using the Compressed Sparse Row (CSR) format. Building upon the optimized single-GPU kernel developed in the previous deliverable. We compare a basic implementation based on one-dimensional cyclic partitioning with a more advanced implementation employing two-dimensional matrix partitioning. The evaluation is conducted on a four-GPU system and measures communication and execution time, FLOPs, scalability, speed-up, efficiency, and memory footprint per rank. These findings highlight the importance of balancing communication costs and partitioning overhead in distributed SpMV implementations.

Index Terms—SpMV, Multi-GPU, CSR, Strided access pattern

I. INTRODUCTION

Sparse matrix-vector multiplication (SpMV) is a common operation in many scientific applications. As the name suggests, SpMV consists in the multiplication of a sparse matrix – a matrix which elements are mostly zero – and a dense vector. SpMV is known to be a bottleneck in scientific computational applications and to not use efficiently the processor performance [3], making it memory bound due to its irregular access to memory. [2]

The goal of this paper is to improve on the previous findings. In the last deliverable, it was found that a kernel that performs SpMV on a CSR-stored matrix that uses a strided access pattern with 1024 threads per block and no shared memory was the best performing. As such, we will take that kernel and its configuration and use it on a multi-GPU system. We will then analyse the performances in terms of communication and execution time, FLOPs, speed-up, efficiency and memory footprint of each rank. Additionally, we will also look at the scaling of the system and how load balance and the matrix structure affects the performances.

II. METHODOLOGY

In order to evaluate the system properly a variety of configurations were tested. Two implementations, a basic and an advanced, of the system were developed in order to do so. The basic version has a straightforward approach to the problem. Only the root process reads the matrix file and it then splits it, with 1D cyclic partitioning, to the other processes, then every process performs SpMV on their submatrix and a final gather is performed in order to reconstruct the global solution. The advanced version, instead, works on a 2D grid partitioning and each process reads the file in parallel. This cuts down the communication time, but increases the distribution complexity. Additionally, this version divides the load balance on different

strategies based on the number of non-zeroes per process, or the communication volume between processes.

A third version that utilizes NCCL¹ was developed on top of the advanced version, but the results were not relevant and as such will not be reported in this paper.

Lastly, in order to compare this parallel system, a sequential CPU algorithm² was developed as the baseline.

A. Data distribution

In the basic version, the data distribution is managed by the root process. It reads the matrix from a file and then computes how to split the data to the other processes. In order to decide to which process to assign the i -th row, it solves the following equation: $owner[i] = i \bmod p$, where p is the number of processes.

The advanced implementation adopts a two-dimensional data decomposition [1]. First, the MPI processes are arranged into a 2D Cartesian grid. The global matrix is then partitioned independently along the row and column dimensions. Each process is assigned one contiguous interval of rows and one contiguous interval of columns, resulting in a rectangular submatrix. A matrix entry is stored by the process whose row interval contains its row index and whose column interval contains its column index. Additionally, in order to have load balancing, the row and column partitions are computed independently with a weighted contiguous partitioning. The three load balancing modes are: uniform (equal weight partitioning), balanced on the nnz per process and, lastly, balanced on the communication volume to each rank.

B. Communication

As stated, in the basic implementation, the data is read only by the root process and is then communicated to the other processes. Meaning that, since the matrix is stored in a CSR format, we have to communicate around $O(2 \times nnz/p)$ values to each process. Since each process stores complete rows of the matrix, global row indexing information is not needed during the computation. After computing the SpMV for their submatrix, each process communicates the results back to the root which will reconstruct the final solution and redistribute it.

In the advanced implementation, each process reads its local matrix partition directly from the input file, eliminating the need for the root process to distribute the matrix. Communication is primarily required to exchange global indexing

¹<https://developer.nvidia.com/nccl> - Last visited: July 29, 2026

²<https://github.com/HicrestLaboratory/GPU-Computing26/blob/main/multiGPU-SpMV/SpMV.cpp> - Last visited on: July 29, 2026

TABLE I
SYSTEM HARDWARE SPECIFICATIONS

Device	Metric	Value
NVIDIA A30	Memory BW	933 GB/s
	FP32 Perf.	10.3 TFLOPS
	Threads/Block	1,024
	Global Memory	23.49 GB

information and to assemble the final solution vector. Consequently, the initial communication overhead is significantly reduced compared to the basic implementation at the cost of the increase in complexity of the data distribution part.

For both versions, the dense vector is communicate from the root processes to all others.

C. Configurations

The matrix is stored in a CSR format and is accessed with a strided pattern. The kernel configuration used is of 1024 threads per block and no shared memory is used. In order to test the multi GPU system, we decided to use up to 4 GPUs, assigning one to each MPI³ process.

D. Environment

Hardware

The experiments were run on the UNITN DISI Cluster, which utilizes four NVIDIA A30⁴ as the GPUs.

Software

All the programs were compiled using the following configuration `nvcc -ccbin mpicc -lnvidia-ml -lmpi -O3 -use_fast_math`.

The version of *gcc* is 12.3.0, of *OpenMPI* is 4.1.5 and of *CUDA* is 12.5.0. The data-type used to store the non-zeroes values is Float32.

E. Measurement methodology

Each process has been timed and measured across 5 timed iterations with 1 previous warm-up cycle. Each measurement is calculated as follow:

- **Communication time**, as the maximum time between the processes [s];
- **Execution time**, as the maximum time between the processes [s];
- **Speedup**, as $\frac{Latency_1}{Latency_2}$, where $Latency_1$ is the latency of the program being run on a single CPU⁵ and $Latency_2$ is the latency of the current configuration;
- **Efficiency**, as $\frac{Speedup}{\#processes}$;
- **Floating point operations per second (FLOPS)**, measured as the standard $\frac{2 \times nnz}{10^9 \times exec_time}$ [GFLOPS];
- **Memory footprint per rank**, measured as the total number of bytes allocated [MB].

To analyse **strong and weak scalability**, we randomly generate three $n \times n$ matrices with different n . The middle one

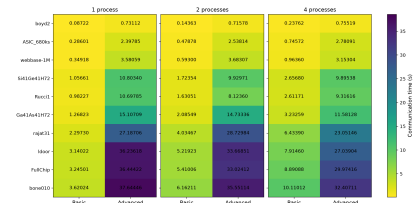


Fig. 1. Communication time (s)

has $n = 2^{19}$ which then was halved ($n = 2^{18}$) and doubled ($n = 2^{20}$) for the other matrices. All these matrices have an average nnz per row of 6.

The final measurement for each metric is obtained by computing the average value of each measure for the same matrix and same number of processes.

For reproducibility, the random dense vector has been set to have all values equal to 1.0.

III. DATASET

The experiments were tested against 10 different matrices sourced from SuiteSparse Matrix Collection⁶, with different regimes in size and sparsity structure. Table II provides some characterization of the matrices. These matrix also will help us analyse the results in terms of load balancing.

TABLE II
OVERVIEW OF THE MATRICES USED

#	Matrix	Dimensions ⁷	nnz	nnz/row	Type	Sym.
1	FullChip	2,987,012	26,621,990	8.9	Unstr.	No
2	Ga41As41H72	268,096	18,488,476	69.0	Unstr.	Yes
3	Si41Ge41H72	185,639	15,011,265	80.9	Unstr.	Yes
4	bone010	986,703	47,851,783	48.5	Diag.	Yes
5	ldoor	952,203	42,493,817	44.6	Unstr.	Yes
6	rajat31	4,690,002	20,316,253	4.3	Diag.	No
7	ASIC_680ks	682,712	1,693,767	2.5	Unstr.	No
8	Rucci1	1,977,885 ×109,900	7,901,068	4.0	Unstr.	No
9	boyd2	466,316	1,500,397	3.2	Unstr.	No
10	webbase-1M	1,000,005	3,105,536	3.1	Unstr.	No

IV. RESULT

Figure 1 shows the communication time for the basic and advanced implementation.

Figure 2 shows the execution time for the basic and advanced implementation.

Figure 3 shows the FLOPs for the basic and advanced implementation.

Figure 4 and 5 show the speed-up and efficiency for both implementations.

Figure 6 shows the breakdown of the memory footprint per number of ranks. Figure 7 does the same but for the advanced implementation.

³<https://www.open-mpi.org/> - Last visited on: July 29, 2026

⁴<https://www.nvidia.com/content/dam/en-zz/Solutions/data-center/products/a30-gpu/pdf/a30-datasheet.pdf> - Last visited on: July 29, 2026

⁵<https://github.com/HicrestLaboratory/GPU-Computing26/blob/main/multiGPU-SpMV/SpMV.cpp> - Last visited on: July 29, 2026

⁶<https://sparse.tamu.edu/> - Last visited on: July 29, 2026

⁷Only one dimension reported if the matrix is square

V. DISCUSSION

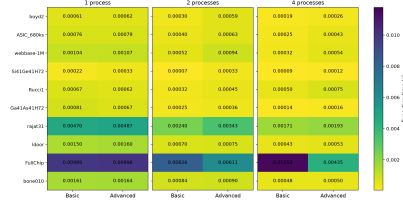


Fig. 2. Execution time (s)

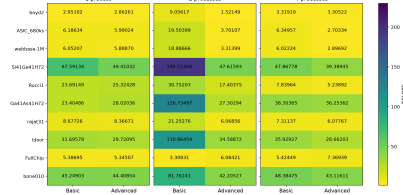


Fig. 3. FLOPS (GFLOPS)

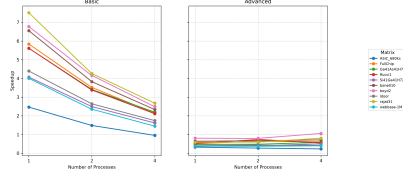


Fig. 4. Speed-up

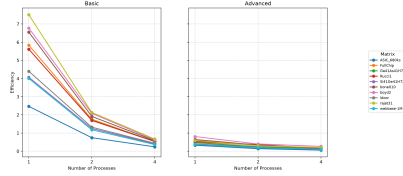


Fig. 5. Efficiency

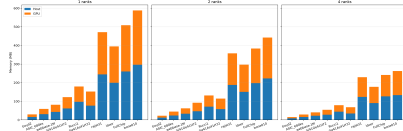


Fig. 6. Memory footprint for 1D partitioning

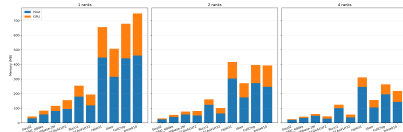


Fig. 7. Memory footprint for 2D partitioning

Let's first discuss the timings. The first thing to note from Fig. 1, is how the advanced version actually takes longer to perform the data partitioning part of the algorithm. This is to be expected as the overhead operations to perform the splitting are much more complicated than the basic version and as such the benefit only appear for bigger number of processes. In fact, we can see that as the number of processes increases, the time decreases. On the contrary, for the basic version, the communication time increases with the number of processes available. This is explained as the waiting time for a single process to communicate to multiple other processes increase with the number of processes. In this latter scenario, the processes that are not receiving the data split are not being used at their full capacity as they need to wait for the other processes to finish. Furthermore, since the execution time are essentially equal between the two versions, we can deduce that most of the performance improvement originates from reducing the communication overhead. In these implementations each MPI process has been assigned one GPU, so the advanced version would require more GPUs to pay off its overhead for the distribution and thus being able to surpass the basic implementation.

This final idea would suggest that the advanced version is more suitable for large-scale executions. This can also be deduced by looking at Fig. 4. We can see how, in the basic version, the speed-up decreases as the number of processes increase, and how this trend does not hold for the advanced version. In this latter version, the speed-up seems to remain more or less constant, with only a subtle increase with the number of processes. Regardless, since the starting speed-up of the basic implementation is much higher than the advanced one, in this experiment, the basic version still outperforms the advanced version.

For the efficiency, we can see in Fig. 5, a steep drop with the increase of the number of processes in both implementations. This is expected because SpMV is a memory-bound problem.

The reduction in memory footprint per rank with increasing process count is expected, as the global matrices are partitioned among more MPI processes and, consequently, each rank stores a smaller fraction of the overall problem. The advanced implementation consistently exhibits a slightly larger memory footprint due to the temporary buffers and additional data structures required during the recursive partitioning process. This overhead is not as very significant as it comes primarily from temporary buffers host-side which are used in the initial state of the matrix partitioning. Additionally, in the advanced version, the memory allocated on the GPU is less than in the basic implementation. This is important as the GPU has often much less memory available compared to the host.

Lets look at the work load balance of the advanced version. From Fig. 8 we can see that the majority of the modalities have similar results in term of balancing the weights and that these results are between 1.0 and 1.1 meaning that, at worst, there is a difference of balance of 10% between the maximum

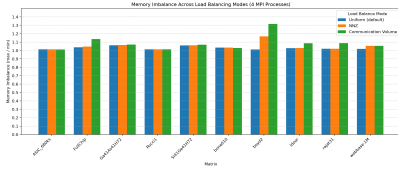


Fig. 8. Memory imbalance for different weight functions (Number of processes set to 4)

and minimum workload amount, and at best there is equal distribution. However, the default weighting function (where every weight is set to be equal to 1) performs equally or better than the other modes. This would suggest that, for the matrices evaluated in this study, load balancing is not too important, as their shape and number of non zeroes naturally split the workload evenly.

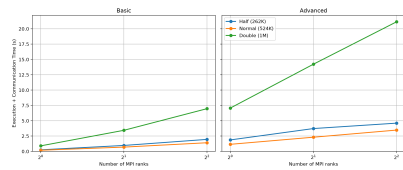


Fig. 9. Strong Scalability

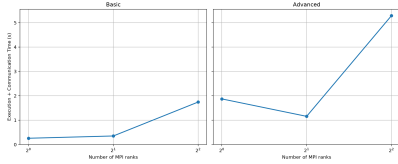


Fig. 10. Weak Scalability

Finally, we analyse the scalability of each version. From Fig. 9 and Fig. 10 we can deduce that the advanced version does not scale as efficiently as the basic version. For strong scalability, we can see how the advanced version not only increases with the number of processes, but it also grows more rapidly than the basic implementation, indicating a poor ability of decreasing time with the addition of more processes. Similarly, the weak scalability is also worse for the advanced version. However, we can see an inflection point when the number of processes is equal to 2 for the advanced version, while the basic version grows steadily. Both of the versions scalability worsen rapidly when reaching 4 processes.

VI. CONCLUSION

From the results of this paper we can deduce that a more basic approach, meaning one with a root process that distributes with 1D cyclic partitioning the matrix to the other processes, outperforms a more advanced algorithm if these algorithms are being run on 4, or less, GPUs and therefore 4 processes. The advanced algorithm does not regain its cost in the span of processes analysed. This is expected as building a 2D partitioning where every process has to read the file is quite

the overhead. The results also show how the biggest hurdle in SpMV operations is the initial loading and distribution of the matrix and how the number of GPU employed to solve this problem does not affect it.

Future works could explore how these systems would operate on different conditions; if the advanced version would outperform the basic implementation if it was being run on more devices or even bigger matrices. More importantly, however, is researching how to efficiently load and distribute the matrix to every process as it would be the most beneficial improvement due to its impact on the time execution of the system.

REFERENCES

- [1] Erik G. Boman, Karen D. Devine, and Sivasankaran Rajamanickam. Scalable matrix computations on large scale-free graphs using 2d graph partitioning. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, SC '13*, New York, NY, USA, 2013. Association for Computing Machinery.
- [2] Genshen Chu, Yuanjie He, Lingyu Dong, Zhezhao Ding, Dandan Chen, He Bai, Xuesong Wang, and Changjun Hu. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing, HPDC '23*, page 115–128, New York, NY, USA, 2023. Association for Computing Machinery.
- [3] Samuel Williams, Leonid Oliker, Richard Vuduc, John Shalf, Katherine Yelick, and James Demmel. Optimization of sparse matrix-vector multiplication on emerging multicore platforms. In *SC '07: Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, pages 1–12, 2007.